NEXTECH / PROYECTO FINAL

Punto de venta de tecnología

Documentación técnica y evidencia de cumplimiento

Versión 2.0 · Revisión: 6 de octubre de 2026

Institución

Instituto Tecnológico Superior de Irapuato

Materia

Desarrollo de Apps Web

Docente

Francisco Hunahpu Sahagun González

Grupo

Ing. Informática

Integrantes del equipo

Soto Saavedra Erwin Emmanuel
Aguirre Sierra Juan Damián
Crisanto Vázquez Sofía
Rosales Alarcón Amairani
Gutiérrez Rocha Kevin Yahir
Rivera Alvarado Itzel Daniela

Fecha de entrega

6 de noviembre de 2026

Enlace al repositorio Git

https://github.com/EESH02011/NEXTECH-

Sitio previsto (pendiente de despliegue)

https://mamawevo.com/

Datos y enlaces proporcionados por el equipo; campos editables.

Dominio de publicación previsto. Despliegue y pruebas en servidor pendientes.

Índice

1. Portada y control del documento 1

Índice 2

2. Resumen ejecutivo 3

3. Problema, objetivos y alcance 4

4. Modelo de negocio y contexto de uso 5

5. Planeación y cronograma 6

6. Requerimientos y criterios de aceptación 7

7. Arquitectura y justificación técnica 9

8. Diseño UX/UI y bocetos 10

9. Diseño de base de datos 13

10. Diseño de API 15

11. Módulos y lógica de desarrollo 17

12. Seguridad 21

13. Pruebas y validación 22

14. Despliegue e instalación 25

15. Resultados, limitaciones y conclusiones 26

16. Anexos y trazabilidad 27

2. Resumen ejecutivo

NexTech atiende una necesidad simulada de una tienda de tecnología: centralizar catálogo, cuentas de clientes, compras e inventario para evitar registros dispersos, errores en cantidades y falta de seguimiento. Su propuesta combina una tienda digital orientada a compradores de componentes y periféricos con una administración que permite controlar el proceso de venta.

La aplicación utiliza HTML, CSS y JavaScript en el navegador, una API REST en PHP y una base relacional. El servidor es responsable de las validaciones, autenticación, permisos, importes y movimientos. MySQL es el destino de instalación en Hostinger; SQLite se utiliza en la demostración local y en las pruebas aisladas. El navegador nunca recibe credenciales de base de datos.

La versión entregada contiene 300 modelos de productos distribuidos inicialmente en 15 categorías, dos roles, 18 tablas, 56 rutas REST, cuatro módulos CRUD administrativos y dos reportes con gráficas. El proceso de compra incluye carrito persistente, reserva de existencias, desglose de impuesto incluido, folio único, seguimiento y cancelaciones autorizadas. Productos, categorías, proveedores y usuarios conservan su historial mediante baja lógica.

En esta revisión se corrigieron faltantes de la entrega previa: rutas REST con verbos apropiados, relaciones y restricciones de base de datos, normalización de especificaciones, módulos de administración, variables de entorno, movimientos, reportes, gráficas y documentación. Se aprobaron 23 grupos de pruebas automáticas y se comprobó la interfaz en escritorio, tableta y teléfono.

El resultado es demostrable localmente. No se ha desplegado en la cuenta de Hostinger del usuario ni se han realizado cobros reales. La acreditación de historial previo de Git y aportes de integrantes permanece pendiente de evidencia auténtica. Los datos académicos fueron proporcionados por el equipo y se incluyen en la portada; el repositorio y el dominio previsto se incluyen y permanecen editables.

Tabla 1. Control del documento

Versión	Fecha	Responsable	Cambio
1.0	Versión previa disponible	Sin atribución de equipo aportada	Tienda inicial construida en este chat.
2.0	06/10/2026	Asistencia técnica automatizada; revisión del equipo pendiente	Auditoría, ampliación funcional, pruebas y documentación.

3. Problema, objetivos y alcance

3.1 Planteamiento del problema

Una tienda que administra productos y pedidos por mensajes o archivos separados puede ofrecer unidades que ya no tiene, calcular totales inconsistentes y perder el seguimiento del cliente. La necesidad planteada en este proyecto es disponer de un sistema web centralizado para una tienda de tecnología con catálogo verificable y controles de punto de venta. El escenario es académico; no se afirma que se haya realizado un estudio de campo con un comercio real.

3.2 Objetivos verificables

Tabla 2. Objetivos y comprobación

Objetivo	Evidencia
General: integrar venta digital y administración de inventario con persistencia y seguridad básica.	Recorrido cliente → API → transacción → pedido e inventario.
Permitir consultar 300 modelos y localizar laptops, marca o SKU.	Catálogo inicial; 12 laptops dentro de Computadoras; CP-01/02/14.
Controlar productos, categorías, proveedores y usuarios.	Cuatro CRUD, permisos y baja lógica; CP-09/15/16/17.
Registrar pedidos consistentes y movimientos auditables.	CP-07/08/19/21; tabla inventory_movements.
Consultar ventas e inventario con indicadores.	Dos reportes y dos gráficas desde GET /admin/reports; CP-20.
Documentar instalación, diseño y verificación.	README, SQL, diccionario, API, capturas y este PDF.

3.3 Alcance y exclusiones

Incluye cliente y administrador; catálogo, perfil, direcciones, favoritos, carrito, pedidos, inventario, proveedores, configuración, auditoría y estadísticas. Se admiten pagos de demostración, transferencia manual y conectores de Stripe/Mercado Pago sujetos a configuración. Proveedores es un directorio relacionado con productos, no un módulo de órdenes de compra.

Fuera de esta versión: facturación CFDI, reembolsos externos automáticos, entrega automática de licencias, cotización o compra de guías, múltiples sucursales, arqueo de cajas, tasas tributarias por producto y pruebas de carga. No se almacena información de tarjetas. El catálogo inicial usa importes ilustrativos y stock cero para instalación; la base de demostración contiene cantidades ficticias claramente identificadas.

4. Modelo de negocio y contexto de uso

Tabla 3. Modelo de operación

Elemento	Aplicación en NexTech
Segmento	Personas que buscan computadoras, componentes, periféricos y tecnología; administradores de una tienda especializada.
Necesidad	Encontrar productos y mantener una relación consistente entre pedido, pago y existencias.
Propuesta de valor	Catálogo con fuentes por producto, autoservicio de compra y seguimiento; control administrativo centralizado.
Proceso	Explorar → carrito → autenticar → dirección y pago → pedido → confirmar → preparar → enviar → entregar.
Actores	Cliente compra y consulta lo propio. Administrador gestiona catálogo, cuentas, inventario, estados, reportes y configuración.
Recursos	Hosting PHP/MySQL, dominio/HTTPS, correo, cuentas de pasarela, abastecimiento, inventario confirmado y personal de atención.
Costos	Alojamiento, dominio, correo, comisiones del proveedor de pago, logística, adquisición de mercancía y mantenimiento. No se aportaron cotizaciones.
Sostenimiento	Margen sobre venta de mercancía y posibles cargos de envío. El sistema no calcula rentabilidad porque no registra costos de compra.
Indicadores	Ventas confirmadas por periodo, número de pedidos, existencias por categoría, productos con stock bajo y estados de entrega.

Flujo de valor y control

La tienda obtiene un pedido identificable y una reserva de inventario; el cliente obtiene un resumen calculado en el servidor y seguimiento desde su cuenta. Las integraciones de cobro solo confirman pagos después de verificar la notificación con el proveedor. La demostración se separa de las ventas reales en el dashboard y en los reportes.

5. Planeación y cronograma

No se proporcionó un cronograma original del equipo ni evidencias de responsables históricos. La tabla siguiente documenta el plan de regularización de esta revisión, establecido y ejecutado el 06/10/2026. No representa una planeación retrospectiva inventada del desarrollo anterior. El equipo debe adjuntar su cronograma original si la evaluación exige acreditar ese proceso.

Tabla 4. Plan de regularización con entregables

Fase / fechas	Responsable	Entregable y evidencia	Estado
Análisis · 06/10 → 06/10	Asistencia técnica; validación del equipo	Comparación de los 35 requisitos y alcance POS; matriz de cumplimiento.	Realizado
Diseño · 06/10 → 06/10	Asistencia técnica; validación del equipo	Modelo relacional, rutas REST y módulos adicionales; esquema y diagramas.	Realizado
Backend · 06/10 → 06/10	Asistencia técnica	CRUD, RBAC, relaciones, movimientos e informes; código y pruebas.	Realizado
Frontend · 06/10 → 06/10	Asistencia técnica	Formularios, listas paginadas, reportes y gráficas; capturas.	Realizado
Pruebas · 06/10 → 06/10	Asistencia técnica	23 grupos y revisión visual en 3 tamaños. JSON de resultados.	Realizado
Entrega técnica · 06/10 → 06/10	Asistencia técnica	Código, SQL, fuentes de documentación y PDF.	Preparada
Cierre académico · 06/11/2026	Equipo NexTech	Portada y enlaces completados; aportes e historial original pendientes.	A cargo del equipo
Despliegue · por definir	Responsable del servidor	Instalación y pruebas en MySQL/Hostinger y pasarelas de prueba.	Pendiente de acceso

Tabla 5. Seguimiento estimado frente a resultado

Fecha prevista / real	Entregable esperado	Resultado y desviación
06/10/2026 / 06/10/2026	Completar tres CRUD mínimos y reportes	Cuatro CRUD y dos reportes disponibles; sin desviación en esta revisión.
06/10/2026 / 06/10/2026	Guardar formularios desde el navegador	Se detectó colisión de name=id con form.id. Se cambió a getAttribute y se verificó el guardado.
06/10/2026 / 06/10/2026	Evidencias de pruebas y PDF	23 grupos aprobados; PDF con portada editable y anexos.
Sin fecha acordada / pendiente	Historial Git y colaboración previa	No existía repositorio local. Se inició uno desde la revisión; no sustituye evidencias anteriores.

Dependencias, riesgos y contingencias

El esquema y los permisos preceden a los CRUD; los pedidos preceden al reporte de ventas; la configuración comercial precede al cobro real. Riesgos: falta de credenciales del hosting, diferencias entre SQLite y MySQL, servicios de pago/correo sin configurar y ausencia de evidencias académicas originales. Contingencia: demostración local aislada, SQL reproducible, pruebas de rollback y documentación explícita de pendientes. Antes de producción: respaldo, instalación de ensayo y compra de prueba completa.

6. Requerimientos y criterios de aceptación

Tabla 6. Requerimientos funcionales

ID	Función	Aceptación	Prueba
RF-01	Autenticar y autorizar	Registro/login/logout; cliente no accede a /admin.	CP-03,16,23
RF-02	Explorar catálogo	Buscar, filtrar, ordenar y paginar productos reales almacenados.	CP-01,02,14
RF-03	Gestionar productos	Crear, leer, editar y desactivar con SKU único y validación.	CP-09,17
RF-04	Gestionar categorías	CRUD y bloqueo de baja con productos activos.	CP-15,17
RF-05	Gestionar proveedores	CRUD y relación opcional con productos; baja protegida.	CP-15,17
RF-06	Gestionar usuarios y roles	CRUD, hash, baja lógica y protección administrativa.	CP-16
RF-07	Gestionar cuenta	Perfil, direcciones, favoritos y recuperación.	CP-04,05,12
RF-08	Comprar	Carrito con cantidades válidas, total del servidor, folio único.	CP-06,07,19
RF-09	Controlar inventario	Reserva y devolución con movimientos dentro de transacción.	CP-08,19,21
RF-10	Seguir pedidos	Historial privado y estados con guía al enviar.	CP-08,10
RF-11	Reportar	Ventas por fechas/tipo e inventario; dos gráficas y CSV.	CP-20 + interfaz
RF-12	Configurar y auditar	Registrar operaciones; bloquear venta real incompleta.	CP-09,11,19

Tabla 7. Requerimientos no funcionales

ID	Condición	Validación / límite
RNF-01	Seguridad básica	Hash, CSRF, consultas preparadas, sesión y RBAC probados; no es una auditoría de penetración.
RNF-02	Persistencia e integridad	18 tablas relacionadas, FK y CHECK; rollback probado. MySQL real pendiente.
RNF-03	Respuesta y rendimiento	Paginación acotada a 100 registros; índices. No hay medición de concurrencia/carga.
RNF-04	Compatibilidad	PHP 8.2+; diseño revisado en escritorio, 768 y 390 px. Otros navegadores no certificados.
RNF-05	Usabilidad	Etiquetas, navegación, confirmaciones y estados de carga/error/vacío.
RNF-06	Mantenibilidad	Capas separadas, variables de entorno, scripts reproducibles y documentación de API.
RNF-07	Disponibilidad	Depende del hosting. Sin SLA ni alta disponibilidad declarados.

Tabla 8. Reglas de negocio

ID	Regla	Implementación
RN-01	No vender productos sin stock suficiente.	Actualización condicional stock>=qty.
RN-02	Cantidad entre 1 y 99 por producto.	Validación servidor y CHECK de carrito.

Tabla 8. Reglas de negocio

ID	Regla	Implementación
RN-03	Total calculado por servidor.	Precios BD, envío y tasa incluida; no se acepta total enviado.
RN-04	Un pedido por clave de idempotencia y usuario.	UNIQUE y recuperación de pedido existente.
RN-05	Venta e inventario son una unidad atómica.	Transacción con líneas, movimiento y auditoría.
RN-06	Devolver stock una sola vez al cancelar.	Transición de estado condicional y movimiento compensatorio.
RN-07	Solo administrador cambia estados comerciales.	require_user(true); cliente solo cancela pendiente permitido.
RN-08	No desactivar relaciones activas ni perder administración.	Conflicto 409 para categorías/proveedores vinculados y acceso propio.
RN-09	Pasarela confirma pagos externos.	Firma más consulta de importe, moneda y referencia.
RN-10	Modo real exige configuración y producto verificado.	Bloqueos en ajustes y checkout.
RN-11	Conservar ventas y sus datos originales.	Baja lógica y detalle/dirección congelados.
RN-12	Separar cifras de demostración y reales.	Filtro de proveedor; anuladas fuera del reporte.

7. Arquitectura y justificación técnica

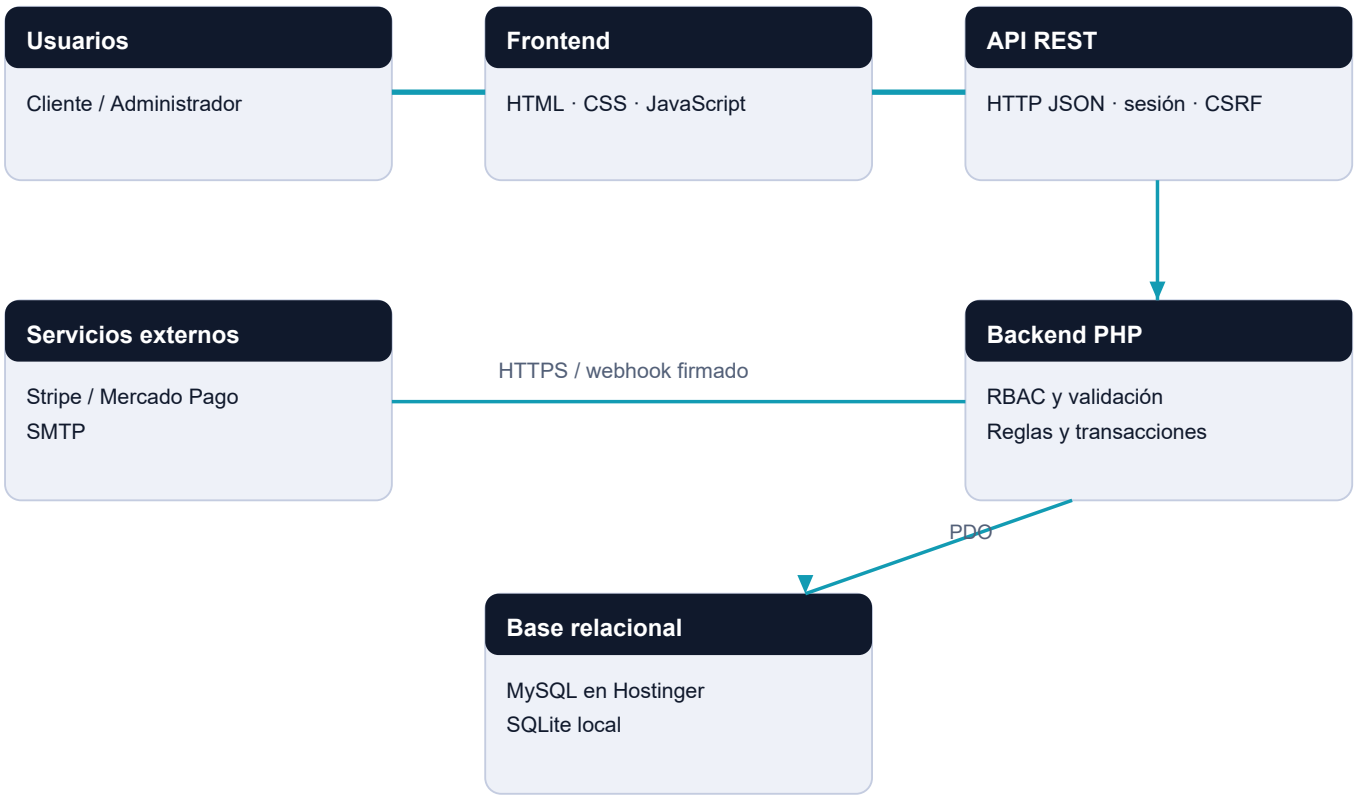


Figura 1. Arquitectura cliente-servidor de NexTech. Diagrama propio de la versión implementada.

El navegador construye la interfaz y realiza validaciones para orientar al usuario. api.php es la entrada HTTP; rest.php resuelve verbo y recurso, lib.php centraliza persistencia y seguridad, admin.php agrupa administración/reportes y payments.php verifica integraciones. La separación facilita mantener el sitio y evita confiar en precios, roles o resultados enviados por el cliente.

PHP se eligió por su compatibilidad con el hosting solicitado y porque permite desplegar sin un proceso Node permanente. PDO concentra consultas preparadas y transacciones. MySQL es el destino del servidor; SQLite permite pruebas reproducibles sin instalar un servicio adicional. Esta diferencia exige validar también en MySQL antes de producción. JavaScript nativo reduce dependencias, aunque el archivo de interfaz conserva funciones extensas y puede dividirse en módulos en una siguiente versión.

Operación de punta a punta: registrar venta

1. El cliente cambia cantidades y confirma dirección, método y términos. 2. JavaScript envía POST /api.php/orders con cookie y CSRF. 3. El servidor valida sesión, método disponible, dirección y clave de idempotencia. 4. Dentro de una transacción consulta el carrito, descuenta stock de forma condicional y calcula importes. 5. Inserta orders, order_items, inventory_movements, order_events y audit; vacía el carrito. 6. Ante error hace rollback. 7. Tras commit, solicita checkout a la pasarela si corresponde y devuelve JSON. 8. La interfaz muestra folio, detalle y seguimiento. Una notificación verificada confirma el pago posteriormente.

Fronteras y límites

public_html es la única carpeta pública. .env, datos, proveedor SMTP y credenciales se mantienen fuera. El carrito de visitante en localStorage es complementario: al autenticar se fusiona con el carrito persistente y el servidor vuelve a validar las existencias. Las solicitudes a servicios externos ocurren fuera de la transacción de reserva para no mantener bloqueos de BD durante llamadas de red.

8. Diseño UX/UI y bocetos

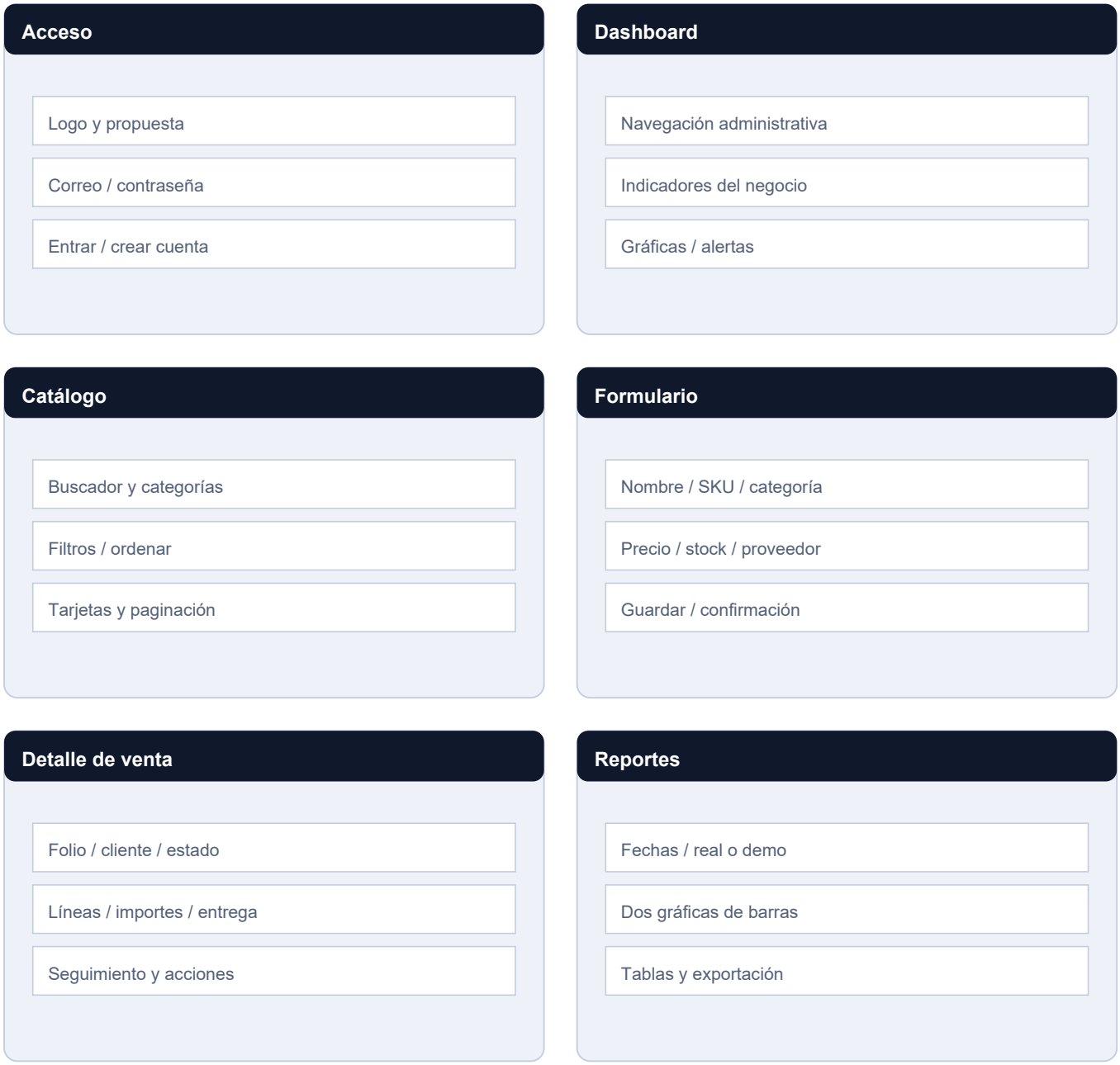


Figura 2. Seis esquemas de navegación y distribución reconstruidos de la versión actual. No se presentan como bocetos históricos aprobados.

La referencia aportada por el usuario orientó una estética gamer: fondo oscuro, acentos morado/cian, búsqueda prominente, menú de categorías, tarjetas de producto y banners. La versión administrativa conserva esa identidad, pero prioriza formularios, tablas, estados y reportes. La tipografía de interfaz es Inter; los controles cuentan con etiquetas y estados visibles. El foco, enlace de salto al contenido y mensajes de error favorecen la navegación básica con teclado.

Comparación entre referencia y resultado

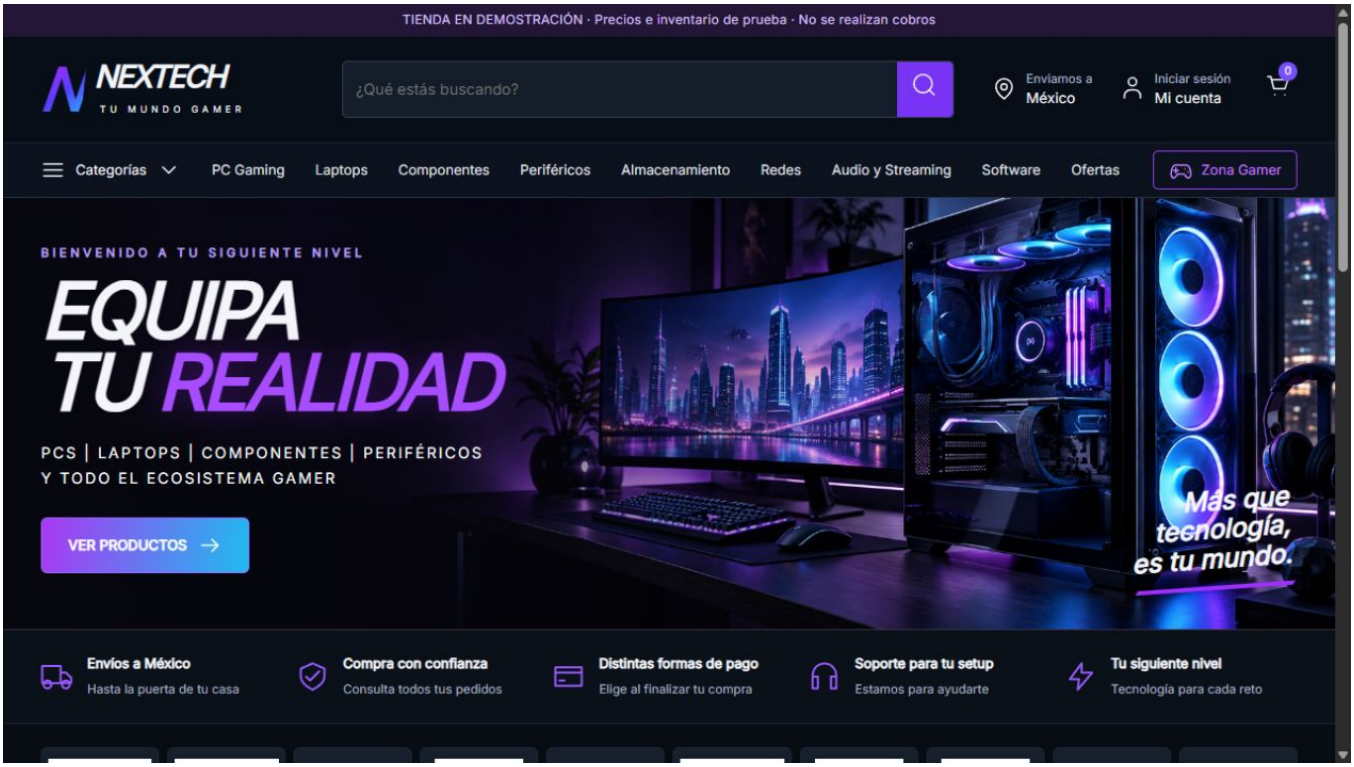


Figura 3. Portada implementada: mantiene el lenguaje visual y conecta búsqueda, cuentas y catálogo.

La referencia era una imagen estática. El resultado añade contenido desde API, disponibilidad, mensajes de demostración, menús funcionales y formularios accesibles. El diseño no es una copia idéntica de cada píxel: adapta tamaños y navegación a pantallas pequeñas y evita presentar ofertas o existencias comerciales sin verificar.

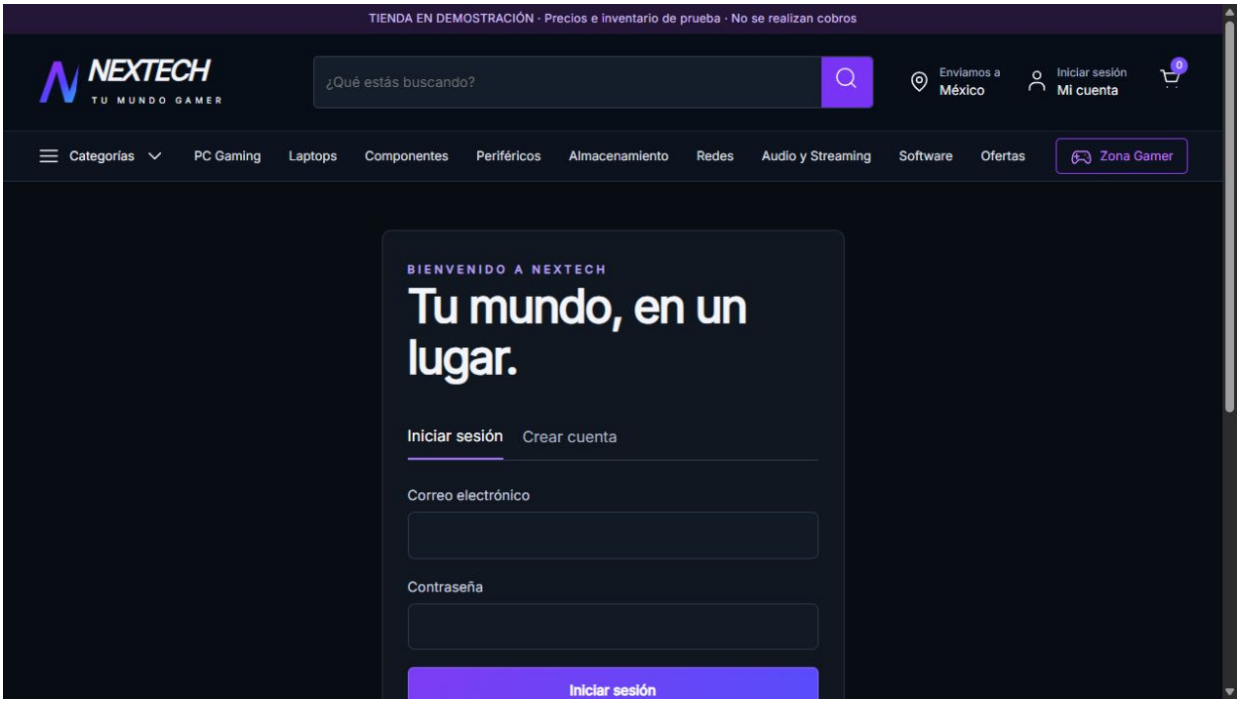


Figura 4. Acceso a la cuenta. Captura de la versión local; sin credenciales reales.

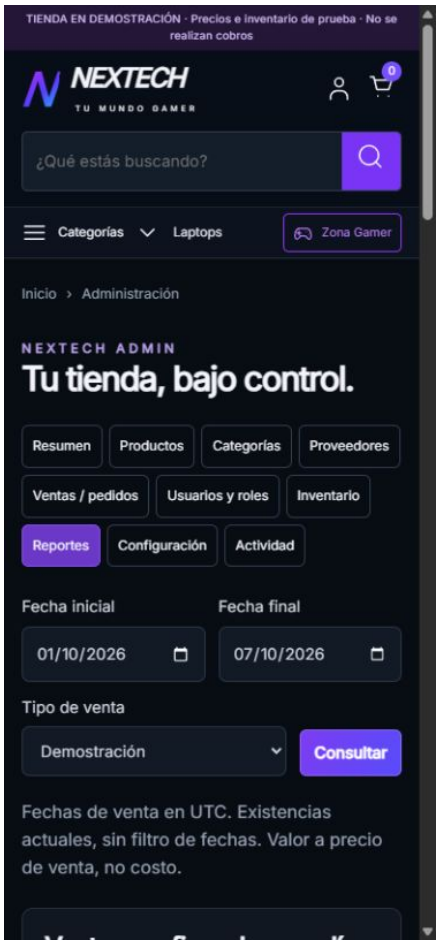


Figura 5. Reporte en teléfono de 390 px; navegación y filtros se reorganizan sin desbordar la página.

9. Diseño de base de datos



Figura 6. Modelo ER principal. 1:N significa un registro padre con cero o varios hijos. PK: clave primaria; FK: foránea; UQ: única.

El modelo completo tiene 18 tablas y se entrega además como Mermaid editable. El núcleo relaciona roles, cuentas, productos, categorías, proveedores, pedidos, detalle y movimientos. Las tablas complementarias soportan carrito, favoritos, direcciones, seguimiento, auditoría, tokens y configuración. El diccionario campo por campo está en el anexo y en docs/database.md.

Tabla 9. Relaciones complementarias

Padre	Hijo / FK	Cardinalidad y propósito
users	cart.user_id; favorites.user_id	1:N; relaciones N:M con products mediante clave compuesta.

Tabla 9. Relaciones complementarias

Padre	Hijo / FK	Cardinalidad y propósito
products	cart.product_id; favorites.product_id	1:N; productos presentes en carritos/favoritos.
users	addresses.user_id; reset_tokens.user_id	1:N; direcciones y tokens privados.
orders	order_events.order_id	1:N; seguimiento cronológico.
users	audit.user_id; inventory_movements.user_id	1:N; responsable de cambios.
settings / rate_limits / payment_events	Sin FK de negocio	Configuración, límites por bucket y claves externas de eventos.

Normalización y conservación histórica

Los datos de usuario dependen de id; el nombre del rol se separa en roles. Productos referencia categorías y proveedores en lugar de duplicar sus datos. product_specs almacena una propiedad atómica por fila con clave (product_id, spec_key), evitando grupos repetitivos. Carrito y favoritos son relaciones con claves compuestas. Estas dependencias permiten una estructura de negocio en 3FN.

orders.address y las columnas históricas de order_items son instantáneas de la venta, no copias vivas del perfil o producto. Se conservan deliberadamente para que cambios posteriores no alteren el comprobante histórico. settings y audit usan JSON para valores tipados y detalles de eventos; la persistencia principal es relacional. Los importes son enteros en centavos para evitar acumulación de errores binarios.

Restricciones, índices y semilla

Se definen FK explícitas, correo/SKU/referencia/idempotencia únicos, CHECK para cantidades y stock, total = subtotal + envío, y balance del movimiento = saldo anterior + cambio. La API agrega validación de correo, fechas, estados y permisos. Índices por categoría/activo, usuario/fecha, fecha/estado y producto/fecha reducen recorridos de consultas frecuentes. Se usa baja lógica para no romper referencias históricas.

database/schema.sql permite crear las tablas; database/seed.sql carga roles, 15 categorías, 300 productos y especificaciones. El instalador ofrece el mismo catálogo mediante seed.php. scripts/demo.php genera cuentas con contraseñas aleatorias y tres pedidos sintéticos solo en una base de desarrollo vacía. Las capturas de reportes usan esos datos, no ventas reales.

10. Diseño de API

Base de las rutas: /api.php. Se usa PATH_INFO para que PHP pueda despachar recursos sin instalar un framework. La aplicación consume rutas REST con GET para consulta, POST para creación, PUT para reemplazo de datos editables, PATCH para cambios parciales/estados y DELETE para retirar relaciones o desactivar recursos. Se mantienen rutas antiguas ?action= como compatibilidad, pero la interfaz usa REST.

Tabla 10. Endpoints principales

Método / ruta	Entrada	Salida / permiso
GET /api.php/bootstrap	Sin parámetros	user, csrf, settings, categories · Público
GET /api.php/products	q, category, brand, stock, featured, offers, sort, page, limit (query)	items, total, page, limit, pages, brands · Público
POST /api.php/auth/register	name (2-120), email, password (10-128), accept=true	user, csrf; 201 · Público
POST /api.php/auth/login	email, password	user, csrf; cookie de sesión · Público
POST /api.php/orders	key (16-80), provider, accept=true, address={recipient,phone,street,colony,city,state,postal_code,notes}	Pedido, folio, total, net_total, tax, tax_rate_bp, items, events; 201 (200 en reintento idéntico) · Cliente autenticado
POST /api.php/admin/products	sku,category,name,brand,description,specs(objeto),image,source_url,price,compare_price,stock,active,featured,verified,supplier_id(opcional); id en PUT	id; 201 al crear, 200 al actualizar · Administrador
PUT /api.php/admin/products/{id}	sku,category,name,brand,description,specs(objeto),image,source_url,price,compare_price,stock,active,featured,verified,supplier_id(opcional); id en PUT	id; 201 al crear, 200 al actualizar · Administrador
DELETE /api.php/admin/products/{id}	id en ruta	ok=true; baja lógica · Administrador
PATCH /api.php/admin/orders/{id}	id en ruta; status, tracking, reason (requerido para anular demo pagada)	ok=true; guarda evento y movimiento si cancela · Administrador
POST /api.php/admin/categories	name,slug (solo creación),icon,position (0-1000),active (0/1)	id,ok=true; 201 al crear, 200 al actualizar · Administrador
PUT /api.php/admin/categories/{id}	name,slug (solo creación),icon,position (0-1000),active (0/1)	id,ok=true; 201 al crear, 200 al actualizar · Administrador
POST /api.php/admin/suppliers	name,email,phone,active (0/1)	id,ok=true; 201 al crear, 200 al actualizar · Administrador
PUT /api.php/admin/suppliers/{id}	name,email,phone,active (0/1)	id,ok=true; 201 al crear, 200 al actualizar · Administrador
POST /api.php/admin/users	name,email,phone,role (admin/customer),active (0/1),password (requerida al crear; opcional al editar)	id,ok=true; 201 al crear, 200 al actualizar · Administrador
PUT /api.php/admin/users/{id}	name,email,phone,role (admin/customer),active (0/1),password (requerida al crear; opcional al editar)	id,ok=true; 201 al crear, 200 al actualizar · Administrador
GET /api.php/admin/reports	start,end (YYYY-MM-DD); scope real/demo	sales, inventory, totals, fechas y notas; alimenta las 2 gráficas · Administrador

Sesión, cuerpos y códigos

GET /bootstrap devuelve estado público y token CSRF. Login y registro regeneran la sesión y el token; las escrituras incluyen X-CSRF-Token y JSON UTF-8. Upload usa multipart y los webhooks tienen firma propia. Los endpoints administrativos comprueban rol en el servidor; los endpoints de cliente filtran por propietario. No se incluyen contraseñas ni claves privadas en respuestas.

Tabla 11. Contrato de errores

Código	Uso
200 / 201	Consulta/actualización correcta; creación de cuentas, productos, módulos administrativos y pedidos.
400 / 422	JSON ilegible / validación del recurso.
401 / 403	Falta de autenticación / CSRF o permiso incorrecto.
404 / 405	Ruta o recurso inexistente / método no permitido.
409	Duplicado, relación activa o conflicto de existencias/estado.
429	Límite de intentos alcanzado.
500 / 503	Error interno genérico / instalación o integración no disponible.

Ejemplo de error: {"error":"No hay suficientes unidades disponibles."}. Paginación: {"items":[],"total":0,"page":1,"limit":20,"pages":0}. Las listas limitan entre 1 y 100 registros por petición. docs/api.md documenta las 56 rutas con parámetros, body, respuestas, errores y permisos; el anexo incluye el inventario completo.

11. Módulos y lógica de desarrollo

Catálogo y productos

Tabla 12. Ficha del módulo Catálogo y productos

Aspecto	Descripción
Objetivo / usuarios	Cliente consulta; administrador mantiene.
Entradas	SKU, nombre, categoría, proveedor, precio, stock, imagen y specs.
Proceso y reglas	Valida campos/relaciones, consulta con filtros, guarda y audita. SKU único; baja lógica.
Datos	products, product_specs, categories, suppliers, audit, inventory_movements.
Endpoints	GET /products; POST/PUT/DELETE /admin/products.
Pantallas	Inicio, catálogo, detalle y administración de productos.
Resultado esperado	Producto consultable y trazable; la búsqueda Laptops muestra 12 modelos de Computadoras.

Categorías y proveedores

Tabla 13. Ficha del módulo Categorías y proveedores

Aspecto	Descripción
Objetivo / usuarios	Administrador.
Entradas	Nombre, clave/orden o correo/teléfono; estado activo.
Proceso y reglas	CRUD con búsqueda y paginación; bloquea baja con productos activos.
Datos	categories, suppliers, products, audit.
Endpoints	GET/POST/PUT/DELETE /admin/categories y /admin/suppliers.
Pantallas	Pestañas Categorías y Proveedores; diálogo de edición.
Resultado esperado	Clasificación y responsables de suministro sin borrar su historial.

Cuentas y roles

Tabla 14. Ficha del módulo Cuentas y roles

Aspecto	Descripción
Objetivo / usuarios	Cliente gestiona lo propio; administrador gestiona cuentas.
Entradas	Nombre, correo, contraseña, teléfono y rol según permiso.
Proceso y reglas	Hash, sesión, CSRF, invalidación por auth_version, baja lógica y protección administrativa.
Datos	users, roles, addresses, favorites, reset_tokens, audit.
Endpoints	/auth/*, /profile, /addresses, /favorites, /admin/users, /admin/roles.
Pantallas	Acceso, registro, perfil, direcciones y Usuarios y roles.
Resultado esperado	Acceso diferenciado, datos propios protegidos y credenciales no expuestas.

Carrito y ventas

Tabla 15. Ficha del módulo Carrito y ventas

Aspecto	Descripción
Objetivo / usuarios	Cliente compra; administrador confirma/prepara/entrega.
Entradas	Cantidades, dirección, método, aceptación, clave única y cambios de estado.
Proceso y reglas	Recalcula total, desglosa impuesto incluido, reserva stock y guarda todos los registros en transacción. Una cancelación válida compensa stock una sola vez.
Datos	cart, orders, order_items, order_events, products, inventory_movements, audit.
Endpoints	/cart, POST /orders, GET /orders/{id}, PATCH /admin/orders/{id}.
Pantallas	Carrito, checkout, historial personal, detalle y Ventas/pedidos.
Resultado esperado	Folio único, importes íntegros y seguimiento persistente.

Inventario, reportes y auditoría

Tabla 16. Ficha del módulo Inventario, reportes y auditoría

Aspecto	Descripción
Objetivo / usuarios	Administrador.
Entradas	Rango de fechas UTC, tipo real/demo, filtros, páginas.
Proceso y reglas	Agrupa ventas confirmadas; excluye anuladas; suma stock actual por categoría; muestra cambios con saldo previo/posterior.
Datos	orders, products, categories, inventory_movements, audit.
Endpoints	GET /admin/reports, /admin/movements, /admin/audit y /admin/dashboard.
Pantallas	Resumen, Reportes, Inventario y Actividad.
Resultado esperado	Dos reportes, dos gráficas dinámicas y exportación CSV; trazabilidad de operaciones.

Configuración e integraciones

Tabla 17. Ficha del módulo Configuración e integraciones

Aspecto	Descripción
Objetivo / usuarios	Administrador; proveedores notifican pagos.
Entradas	Datos comerciales, tasa, envío, políticas, claves .env y firmas.
Proceso y reglas	Bloquea activación incompleta; pagos externos requieren verificación de firma y consulta al proveedor. SMTP solo si configurado.
Datos	settings, orders, payment_events, order_events.
Endpoints	PATCH /admin/settings; POST /webhooks/stripe y /webhooks/mercadopago.
Pantallas	Configuración y detalle del pedido.
Resultado esperado	Operación de demostración segura; activación real condicionada a la configuración.

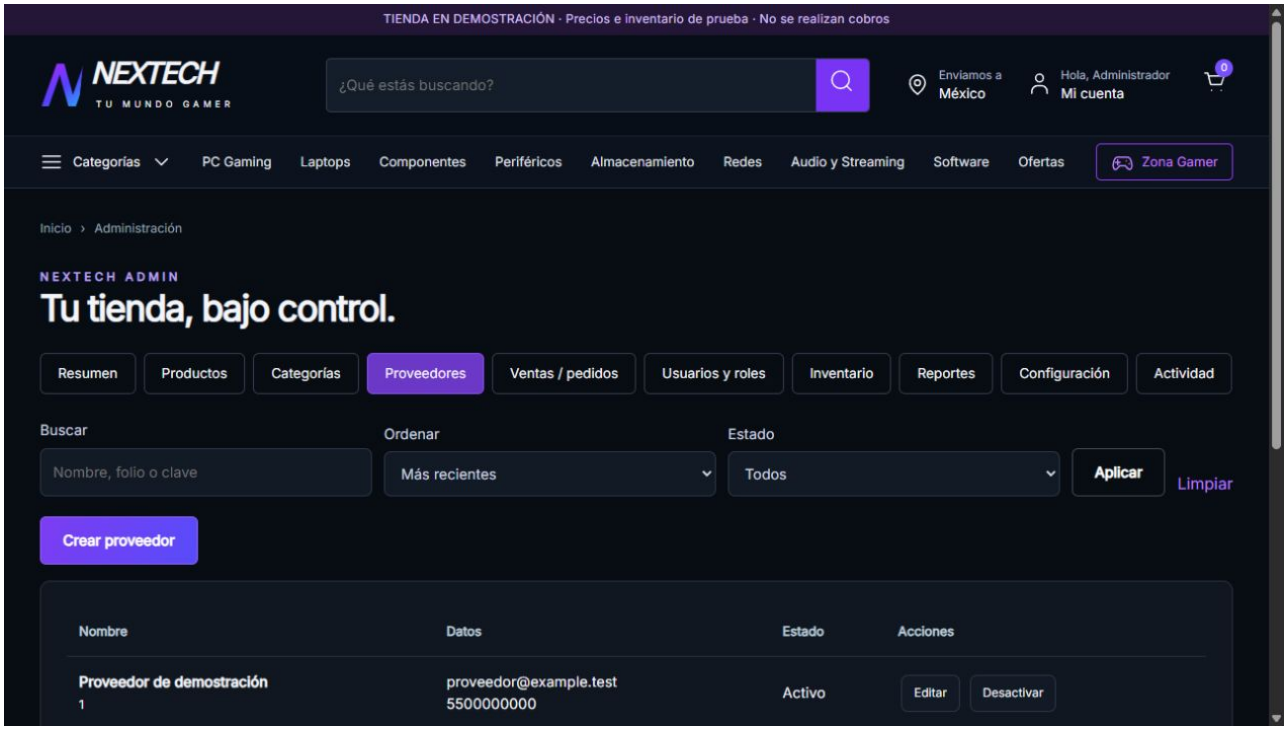


Figura 7. Módulo de proveedores con un registro sintético persistente.

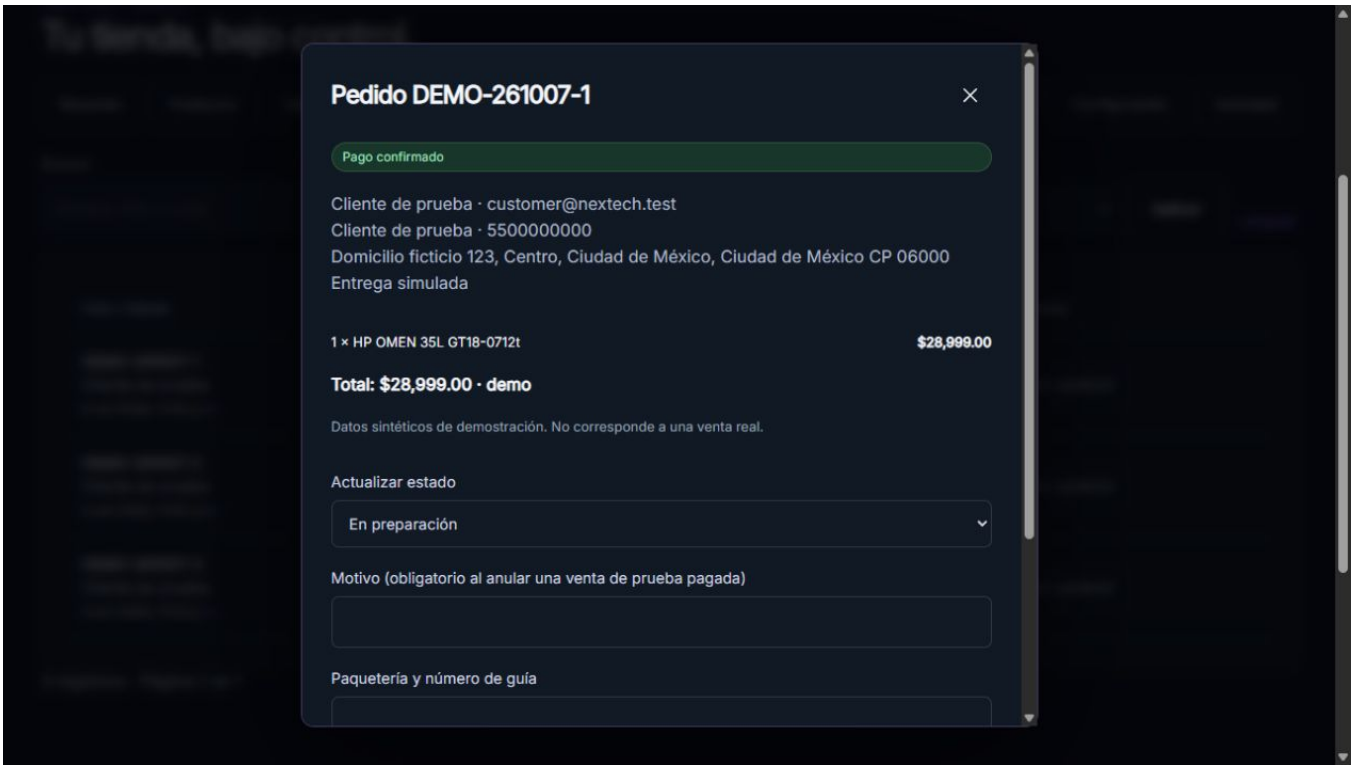


Figura 8. Detalle de una venta de demostración con transición administrativa y motivo de anulación.

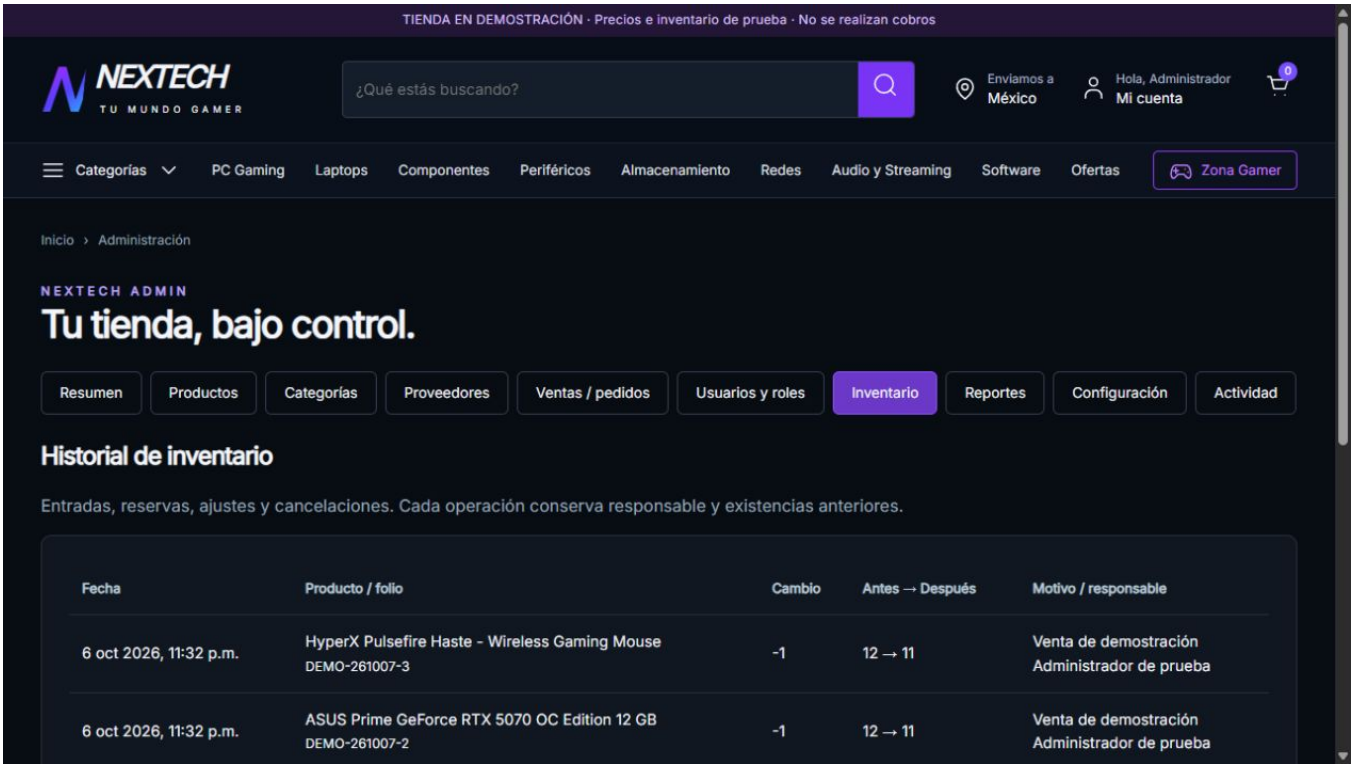


Figura 9. Movimientos de inventario con cambio, saldo anterior/posterior y responsable.

12. Seguridad

Tabla 18. Controles implementados

Control	Implementación y comprobación
Contraseñas	password_hash/password_verify. No se guardan en texto plano ni se retornan a la UI. Tokens de recuperación almacenados como hash, con caducidad y uso único.
Sesiones	Cookies HttpOnly, SameSite=Lax y Secure con HTTPS; modo estricto, regeneración al acceder y 2 horas de inactividad. auth_version invalida sesiones después de cambios.
Permisos	require_user verifica autenticación/rol en backend; cliente solo accede a sus direcciones, carrito y pedidos.
CSRF	Toda escritura de la API exige token de sesión. Webhooks están exentos porque usan firma del proveedor.
SQL e integridad	PDO con parámetros, ordenamiento de lista blanca, FK, CHECK, UNIQUE y transacciones.
Validación / salida	Límites de texto, enteros, direcciones, fechas y URLs. Escapado al generar HTML y respuestas JSON sin SQL ni stack trace.
Carga de imágenes	Solo JPG/PNG/WebP, MIME real y dimensiones comprobadas; máximo 4 MB, nombre aleatorio; uploads bloquea ejecución.
Secretos	Variables .env fuera de public_html y excluidas de Git. Archivo de ejemplo sin claves reales.
Límites de intento	Contadores persistentes para acceso, recuperación y checkout; 429 si excede el umbral.
Cabeceras	CSP y protecciones de tipo, origen/frame en .htaccess. El servidor PHP integrado no aplica .htaccess; deben verificarse en hosting.
Auditoría	Creaciones, estados, ajustes y bajas conservan actor, fecha y detalle; no se registran contraseñas.

Se ejecutaron pruebas de autorización, CSRF, SQL malicioso como término de búsqueda, sesión invalidada, duplicados y errores controlados. No se afirma haber realizado pentesting, análisis exhaustivo de dependencias o pruebas de carga. El HTTPS y los permisos del sistema de archivos deben comprobarse en el servidor destino.

13. Pruebas y validación

La batería test/integration.mjs crea una base SQLite temporal, inicia PHP, ejecuta peticiones reales HTTP y elimina solo su directorio temporal al finalizar. Cada grupo comprueba respuestas y estado persistido. Se ejecutó en PHP 8.4.26 y Node.js 22.21.0. La marca técnica está en UTC: 2026-10-07T05:34:30.372Z; corresponde al 06/10/2026 en México.

Tabla 19. Matriz de pruebas automáticas

ID / caso	Entrada y condición	Esperado / obtenido
CP-01 300 productos y 20 por cada categoría	300 productos, 20 por categoría	300 productos, 20 por categoría; conforme, aprobado.
CP-02 Búsqueda, filtros y orden de precios	Búsqueda real y orden ascendente; inyección sin resultados	Búsqueda real y orden ascendente; inyección sin resultados; conforme, aprobado.
CP-03 Sesiones, CSRF y registro seguro	401 sin sesión; 403 CSRF/rol; 201 registro	401 sin sesión; 403 CSRF/rol; 201 registro; conforme, aprobado.
CP-04 Direcciones y aislamiento entre clientes	Solo el dueño modifica direcciones	Solo el dueño modifica direcciones; conforme, aprobado.
CP-05 Perfil y favoritos persistentes	Rol no escalable; favoritos persistentes	Rol no escalable; favoritos persistentes; conforme, aprobado.
CP-06 Stock insuficiente y validación de cantidades	409 sin stock; rechazo de cantidad negativa	409 sin stock; rechazo de cantidad negativa; conforme, aprobado.
CP-07 Checkout calcula total, reserva stock e impide doble pedido	Total correcto; stock reservado; mismo folio al reintentar	Total correcto; stock reservado; mismo folio al reintentar; conforme, aprobado.
CP-08 Pedidos privados y cancelación devuelve stock una sola vez	404 a otro usuario; devolución única	404 a otro usuario; devolución única; conforme, aprobado.
CP-09 Alta, edición y ocultación de producto con auditoría	Alta/edición/baja con auditoría	Alta/edición/baja con auditoría; conforme, aprobado.
CP-10 Flujo del pedido: pago, preparación, guía y entrega	Transiciones válidas y guía antes de envío	Transiciones válidas y guía antes de envío; conforme, aprobado.
CP-11 Ventas reales bloqueadas sin configuración comercial	Bloquear tienda/pago incompleto	Bloquear tienda/pago incompleto; conforme, aprobado.
CP-12 Recuperación de contraseña con token único e invalidación de sesión	Token de un uso y sesión revocada	Token de un uso y sesión revocada; conforme, aprobado.
CP-13 Webhooks sin configurar no aceptan pagos	No aceptar webhook sin configurar	No aceptar webhook sin configurar; conforme, aprobado.
CP-14 REST: métodos, validación 422, rutas 404 y permisos 403	Verbos correctos y 404/405/403/422	Verbos correctos y 404/405/403/422; conforme, aprobado.
CP-15 CRUD categorías y proveedores con duplicados y baja lógica	Crear/leer/editar/desactivar; duplicado 409	Crear/leer/editar/desactivar; duplicado 409; conforme, aprobado.
CP-16 CRUD usuarios, roles e invalidación de sesiones al dar baja	Usuarios/roles; sesión revocada; acceso propio protegido	Usuarios/roles; sesión revocada; acceso propio protegido; conforme, aprobado.
CP-17 REST productos: creación, edición, proveedor e integridad de categorías	FK proveedor/categoría; baja dependiente bloqueada	FK proveedor/categoría; baja dependiente bloqueada; conforme, aprobado.

Tabla 19. Matriz de pruebas automáticas

ID / caso	Entrada y condición	Esperado / obtenido
CP-18 Paginación, búsqueda y orden administrativos consultan la BD	7 registros por página; búsqueda y orden efectivos	7 registros por página; búsqueda y orden efectivos; conforme, aprobado.
CP-19 Impuesto incluido, movimientos y cancelación administrativa con motivo	Base + impuesto = total; anulación con motivo	Base + impuesto = total; anulación con motivo; conforme, aprobado.
CP-20 Dos reportes: fechas, exclusión de anuladas y separación real/demo	Fechas válidas; demo separada; anuladas excluidas	Fechas válidas; demo separada; anuladas excluidas; conforme, aprobado.
CP-21 Integridad referencial, restricción stock y rollback completo ante fallo de movimiento	Fallo de movimiento revierte stock, pedido y carrito	Fallo de movimiento revierte stock, pedido y carrito; conforme, aprobado.
CP-22 JSON inválido 400 y error interno controlado 500	JSON inválido 400 y fallo BD genérico 500	JSON inválido 400 y fallo BD genérico 500; conforme, aprobado.
CP-23 Logout revoca acceso a la cuenta	Logout quita acceso a pedidos	Logout quita acceso a pedidos; conforme, aprobado.

Tabla 20. Validación de interfaz

Caso	Acción / resultado	Estado
UI-01	Crear categoría visual y desactivarla desde diálogo; aparece en lista y conserva estado inactivo.	Aprobado
UI-02	Guardar producto desde formulario. Se corrigió colisión de form.id con control name=id.	Aprobado
UI-03	Buscar laptop + Computadoras: 12 resultados en administración.	Aprobado
UI-04	Dos reportes con 3 ventas sintéticas y existencias desde la API; capturas guardadas.	Aprobado
UI-05	Reportes en 768 y 390 px. Contenido 753/375 px, dentro del ancho de la página; tablas con scroll local.	Aprobado
UI-06	Capturas de inicio, login, detalle, proveedores e inventario; sin errores de consola al cierre.	Aprobado



Figura 10. Ampliación de la gráfica de ventas del reporte: tres operaciones sintéticas persistidas; no son cobros reales.



Figura 11. Ampliación de la segunda gráfica: unidades actuales por categoría. La captura completa está en docs/evidencias/reportes.png.

Límites de la verificación: no se ejecutó la batería sobre MySQL de Hostinger ni contra pasarelas/SMTP reales. No se midió rendimiento bajo carga y no se probó cada combinación de navegador. Los casos del servidor y la revisión de UI constituyen evidencia acotada, no una garantía absoluta de operación comercial.

14. Despliegue e instalación

14.1 Requisitos y descarga

PHP 8.2+ con PDO MySQL/SQLite, cURL, OpenSSL, mbstring y fileinfo. MySQL 8.0.16+ o MariaDB 10.6+ en Hostinger. Node.js 22+ solo para pruebas. Descargar y descomprimir el ZIP entregado; el repositorio indicado por el equipo es https://github.com/EESH02011/NEXTECH-. Verificar la versión y el acceso antes de descargar. Los archivos PHP, imágenes y PHPMailer ya se incluyen; no se requiere instalar Composer para esta versión.

14.2 Variables y base de datos

Copiar .env.example como .env al mismo nivel que private y public_html. Completar APP_ENV=production, APP_URL=https://mamawevo.com, DB_HOST, DB_PORT, DB_NAME, DB_USER y DB_PASSWORD. Dejar el archivo fuera de la carpeta pública. SMTP_* y claves STRIPE_*/MP_* se agregan únicamente cuando se van a probar esas integraciones. No conservar secretos en Git ni en capturas.

Crear una base vacía. Subir public_html a la raíz pública y private a su lado. Abrir /install.php con la clave private/setup.key; elegir el administrador. El instalador crea tablas y catálogo. Si el proyecto procede de Git sin esa clave, ejecutar php scripts/setup-key.php. La alternativa SQL consiste en importar schema.sql y seed.sql una vez y después usar install.php para crear el administrador. No combinar repetidas cargas de semilla.

14.3 Ejecución y usuarios de prueba

Desarrollo local: APP_ENV=development, APP_URL=http://127.0.0.1:8088, DB_DRIVER=sqlite y DB_PATH con ruta absoluta a una base nueva. Ejecutar php scripts/demo.php y después php -S 127.0.0.1:8088 -t public_html. Las cuentas admin@nextech.test y customer@nextech.test reciben contraseñas aleatorias en private/storage/demo-accounts.json. Este archivo no se publica ni se incluye en Git. Las pruebas automáticas utilizan por defecto el puerto 8089. Las capturas académicas se obtuvieron de una instalación aislada en 8090.

Para probar: establecer PHP_BINARY si PHP no está en PATH y ejecutar node test/integration.mjs. La base usada por la batería es temporal y no modifica el inventario de la tienda. TEST_PORT y TEST_REPORT permiten indicar puerto y archivo de resultados. El esquema se regenera mediante php scripts/export-schema.php.

14.4 Puesta en producción y operación

Activar HTTPS, permisos mínimos de escritura y PATH_INFO para /api.php/recurso. Eliminar install.php y setup.key tras instalar; conservar installed.lock. Verificar que .env/private no son accesibles por web. Confirmar catálogo, precios, stock, tasa aplicable y productos verificados; completar datos comerciales, contacto y políticas. Probar métodos de pago y SMTP en sus entornos de prueba antes de activar ventas reales.

Respaldo: archivos, uploads, .env y MySQL; scripts/backup.php produce un SQL fuera de la carpeta pública. Ensayar restauración. La actualización local anterior cuenta con scripts/upgrade-sqlite.php y respaldos. Una base MySQL existente requiere una migración ensayada; no debe sustituirse por el esquema inicial ni volver a importar la semilla sobre datos comerciales.

15. Resultados, limitaciones y conclusiones

Tabla 21. Resultado frente al mínimo

Elemento	Mínimo solicitado	Versión entregada
Base / tablas	1 base / 6 relacionadas	1 base por instalación / 18 tablas, con FK de negocio
CRUD principales	3	4: productos, categorías, proveedores y usuarios
API / endpoints	REST / 12	REST / 56 rutas por método y patrón
Roles	2	Cliente y administrador, comprobados en servidor
Dashboard	1	Indicadores, tablas, alertas y gráficas
Reglas	5	12 documentadas; cubiertas por pruebas críticas
Reportes / gráficas	2 / 2	Ventas e inventario / ventas diarias y stock por categoría
Transacción / historial	Al menos 1	Checkout, cancelación, cambios administrativos / eventos y auditoría
Pruebas	Casos exitosos y críticos	23 grupos automáticos y 6 comprobaciones de interfaz
Git	Progresivo y colaborativo	Historial iniciado en esta revisión; evidencia anterior no disponible

Pendientes que no pueden acreditarse solo con código

El requisito 33 exige historial de desarrollo progresivo y contribución identificable de integrantes. La carpeta no contenía Git; se creó un repositorio con cambios reales de esta revisión, atribuidos a NexTech automation. No se inventaron commits antiguos ni autores humanos. Para cerrar ese requisito se necesita evidencia auténtica del equipo. Tampoco se recibió la planeación original: este documento identifica el cronograma como regularización actual, y deja al equipo aportar el antecedente real.

Los datos académicos y la fecha de entrega del 6 de noviembre de 2026 se incluyen según la información proporcionada por el equipo. El repositorio proporcionado es https://github.com/EESH02011/NEXTECH- y el sitio previsto es https://mamawevo.com/. El sistema no ha sido publicado en Hostinger y las integraciones reales están pendientes de configurar y probar. El cumplimiento técnico local no equivale a disponibilidad comercial en el servidor.

Conclusiones

La revisión convierte una tienda funcional inicial en una entrega con mayor trazabilidad académica: contratos REST, módulos CRUD adicionales, integridad de base de datos, transacciones con movimientos, reportes y evidencia verificable. La comprobación en navegador fue necesaria para detectar un problema que las pruebas HTTP no mostraban: formularios cuyo control id ocultaba el identificador del formulario.

Las decisiones centrales fueron mantener reglas en el servidor, normalizar especificaciones, preservar instantáneas de ventas y distinguir demostración de operación real. Como aprendizaje técnico observable, las pruebas de rollback muestran por qué venta y existencias deben confirmarse juntas. Las conclusiones personales de los integrantes deben ser redactadas por ellos; no se atribuyen experiencias no proporcionadas. Las siguientes prioridades son validar MySQL/Hostinger, aportar evidencia académica original y evaluar integraciones, carga y operación comercial.

16. Anexos y trazabilidad

16.1 Matriz de cumplimiento general

Tabla 22. Trazabilidad de los 35 requisitos

N.º	Requisito	Resultado	Evidencia
1	Arquitectura mínima	Cumple localmente	public_html → API → private → PDO
2	Cliente-servidor	Cumple localmente	Capas y diagrama de sección 7
3	Base de datos e integridad	Cumple localmente	18 tablas / FK / CHECK / specs normalizadas
4	Modelo / SQL / datos	Cumple localmente	SQL, diccionario, ER y demo.php
5	API REST	Cumple localmente	rest.php: 56 rutas; frontend las consume
6	Formato JSON	Cumple localmente	JSON; CSV/multipart como excepciones
7	Códigos HTTP	Cumple localmente	CP-03/14/15/22; errores diferenciados
8	Tres CRUD	Cumple localmente	Productos, categorías, proveedores, usuarios
9	Autenticación	Cumple localmente	CP-03/12/23
10	Roles y permisos	Cumple localmente	roles + require_user(true); CP-16
11	Autorización de endpoints	Cumple localmente	Cliente aislado; CP-04/08/14
12	Dashboard	Cumple localmente	Resumen con BD, gráficas y stock bajo
13	Buscador	Cumple localmente	CP-02/14; laptops: 12
14	Filtros	Cumple localmente	Categoría, marca, estado y fechas
15	Ordenamiento	Cumple localmente	Catálogo y tablas administrativas
16	Paginación	Cumple localmente	Limit/page/total/pages; CP-18
17	Validaciones	Cumple localmente	Servidor + formularios; CP-06/14
18	Errores comprensibles	Cumple localmente	400/422/409/500 sin SQL; CP-22
19	Reglas de negocio	Cumple localmente	12 RN de sección 6
20	Transacciones	Cumple localmente	Checkout/rollback; CP-07/21
21	Historial	Cumple localmente	audit/order_events/inventory_movements
22	Baja lógica	Cumple localmente	active en cuatro módulos
23	Dos reportes	Cumple localmente	Ventas por periodo e inventario actual
24	Dos gráficas	Cumple localmente	Ventas diarias / unidades por categoría
25	Rangos de fechas	Cumple localmente	start/end y validación; CP-20
26	Diseño responsivo	Cumple localmente	UI-05: 390/768/escritorio
27	Navegación	Cumple localmente	Cliente y administrador con menús

Tabla 22. Trazabilidad de los 35 requisitos

N.º	Requisito	Resultado	Evidencia
28	Estados de interfaz	Cumple localmente	Carga, error, vacío y éxito
29	Confirmaciones	Cumple localmente	Baja y cambio de estado confirmados
30	Seguridad básica	Cumple localmente	Hash, sesiones, CSRF, PDO, RBAC
31	Variables .env	Cumple localmente	.env.example / environment.php / .gitignore
32	Documentación API	Cumple localmente	docs/api.md y 56 contratos
33	Git progresivo y colaborativo	Pendiente de evidencia	Sin evidencia anterior; commits de esta revisión
34	README	Documentado localmente	README técnico, datos académicos y enlaces proporcionados
35	Estructura del proyecto	Cumple localmente	public_html / private / database / docs / test

16.2 Requisitos específicos de punto de venta

Tabla 23. POS y diferenciador

Requisito	Evidencia
Usuarios, roles, productos, categorías, clientes	CRUD y cuentas con dos roles; relaciones en BD.
Inventario, ventas, detalle, historial y reportes	orders, order_items, inventory_movements, audit; módulos administrativos.
Carrito: agregar, cambiar y retirar	PUT/DELETE /cart/items/{id}; validaciones de stock.
Subtotal, impuestos y total	Cálculo servidor; tasa incluida configurable; CP-19.
Folio e historial de ventas	Identificador único y detalle por propietario.
Cancelar bajo permiso especial	Administrador puede anular demo pagada con motivo; pendiente permitido se compensa. Reembolsos externos no automatizados.
Stock bajo y movimiento transaccional	Dashboard y transacción venta/líneas/stock/movimiento; rollback CP-21.
Diferenciador opcional del POS personalizado	Directorio CRUD de proveedores relacionado con productos.

16.3 Inventario completo de API

Tabla 24. Rutas disponibles

Método	Recurso (base /api.php)	Permiso
GET	/bootstrap	Público
GET	/products	Público
GET	/products/{id}	Público
POST	/auth/register	Público
POST	/auth/login	Público
DELETE	/auth/session	Cliente autenticado
POST	/auth/forgot	Público
POST	/auth/reset	Público
PATCH	/profile	Cliente autenticado
PUT	/profile/password	Cliente autenticado
GET	/cart	Cliente autenticado
PUT	/cart/items/{id}	Cliente autenticado
DELETE	/cart/items/{id}	Cliente autenticado
GET	/favorites	Cliente autenticado
POST	/favorites	Cliente autenticado
GET	/addresses	Cliente autenticado
POST	/addresses	Cliente autenticado
PUT	/addresses/{id}	Cliente autenticado
DELETE	/addresses/{id}	Cliente autenticado
GET	/orders	Cliente autenticado
POST	/orders	Cliente autenticado
GET	/orders/{id}	Cliente autenticado
PATCH	/orders/{id}/cancellation	Cliente autenticado
POST	/orders/{id}/payment	Cliente autenticado
GET	/admin/dashboard	Administrador
GET	/admin/products	Administrador
POST	/admin/products	Administrador
PUT	/admin/products/{id}	Administrador
DELETE	/admin/products/{id}	Administrador

Tabla 24. Rutas disponibles

Método	Recurso (base /api.php)	Permiso
GET	/admin/orders	Administrador
PATCH	/admin/orders/{id}	Administrador
GET	/admin/customers	Administrador
GET	/admin/audit	Administrador
GET	/admin/export	Administrador
PATCH	/admin/settings	Administrador
POST	/admin/uploads	Administrador
GET	/admin/categories	Administrador
GET	/admin/categories/{id}	Administrador
POST	/admin/categories	Administrador
PUT	/admin/categories/{id}	Administrador
DELETE	/admin/categories/{id}	Administrador
GET	/admin/suppliers	Administrador
GET	/admin/suppliers/{id}	Administrador
POST	/admin/suppliers	Administrador
PUT	/admin/suppliers/{id}	Administrador
DELETE	/admin/suppliers/{id}	Administrador
GET	/admin/users	Administrador
GET	/admin/users/{id}	Administrador
POST	/admin/users	Administrador
PUT	/admin/users/{id}	Administrador
DELETE	/admin/users/{id}	Administrador
GET	/admin/roles	Administrador
GET	/admin/movements	Administrador
GET	/admin/reports	Administrador
POST	/webhooks/stripe	Proveedor con firma
POST	/webhooks/mercadopago	Proveedor con firma

16.4 Diccionario de datos completo

addresses

Tabla 25. Campos de addresses

Campo	Tipo	Requerido	Clave / valor inicial
id	INTEGER	Sí	PK 1
user_id	INTEGER	Sí	FK users.id
label	VARCHAR(50)	Sí	-
recipient	VARCHAR(120)	Sí	-
phone	VARCHAR(30)	Sí	-
street	VARCHAR(250)	Sí	-
colony	VARCHAR(120)	Sí	-
city	VARCHAR(120)	Sí	-
state	VARCHAR(120)	Sí	-
postal_code	VARCHAR(10)	Sí	-
notes	VARCHAR(500)	Sí	; inicial ''

audit

Tabla 26. Campos de audit

Campo	Tipo	Requerido	Clave / valor inicial
id	INTEGER	Sí	PK 1
user_id	INTEGER	Sí	FK users.id
action	VARCHAR(100)	Sí	-
detail	TEXT	Sí	-
created_at	VARCHAR(30)	Sí	-

cart

Tabla 27. Campos de cart

Campo	Tipo	Requerido	Clave / valor inicial
user_id	INTEGER	Sí	PK 1 FK users.id
product_id	INTEGER	Sí	PK 2 FK products.id
qty	INTEGER	Sí	-

categories

Tabla 28. Campos de categories

Campo	Tipo	Requerido	Clave / valor inicial
slug	VARCHAR(40)	Sí	PK 1
name	VARCHAR(90)	Sí	-
icon	VARCHAR(30)	Sí	-
position	INTEGER	Sí	-
active	INTEGER	Sí	; inicial 1

favorites

Tabla 29. Campos de favorites

Campo	Tipo	Requerido	Clave / valor inicial
user_id	INTEGER	Sí	PK 1 FK users.id
product_id	INTEGER	Sí	PK 2 FK products.id

inventory_movements

Tabla 30. Campos de inventory_movements

Campo	Tipo	Requerido	Clave / valor inicial
id	INTEGER	Sí	PK 1
product_id	INTEGER	Sí	FK products.id
user_id	INTEGER	Sí	FK users.id
order_id	VARCHAR(40)	No	FK orders.id
quantity	INTEGER	Sí	-
before_stock	INTEGER	Sí	-
after_stock	INTEGER	Sí	-
reason	VARCHAR(120)	Sí	-
created_at	VARCHAR(30)	Sí	-

order_events

Tabla 31. Campos de order_events

Campo	Tipo	Requerido	Clave / valor inicial
id	INTEGER	Sí	PK 1
order_id	VARCHAR(40)	Sí	FK orders.id
message	VARCHAR(500)	Sí	-
created_at	VARCHAR(30)	Sí	-

order_items

Tabla 32. Campos de order_items

Campo	Tipo	Requerido	Clave / valor inicial
id	INTEGER	Sí	PK 1
order_id	VARCHAR(40)	Sí	FK orders.id
product_id	INTEGER	Sí	FK products.id
sku	VARCHAR(80)	Sí	-
name	VARCHAR(220)	Sí	-
price	INTEGER	Sí	-
qty	INTEGER	Sí	-
image	VARCHAR(1500)	Sí	; inicial ''

orders

Tabla 33. Campos de orders

Campo	Tipo	Requerido	Clave / valor inicial
id	VARCHAR(40)	Sí	PK 1
user_id	INTEGER	Sí	FK users.id
status	VARCHAR(30)	Sí	-
payment_status	VARCHAR(30)	Sí	-
provider	VARCHAR(30)	Sí	-
payment_id	VARCHAR(190)	No	-
checkout_url	TEXT	No	-
tax_rate_bp	INTEGER	Sí	; inicial 1600
tax	INTEGER	Sí	; inicial 0
net_total	INTEGER	Sí	; inicial 0
subtotal	INTEGER	Sí	-
shipping	INTEGER	Sí	-
total	INTEGER	Sí	-
address	TEXT	Sí	-
idempotency_key	VARCHAR(100)	Sí	-
tracking	VARCHAR(200)	Sí	; inicial ''
notes	TEXT	Sí	-
created_at	VARCHAR(30)	Sí	-
updated_at	VARCHAR(30)	Sí	-

payment_events

Tabla 34. Campos de payment_events

Campo	Tipo	Requerido	Clave / valor inicial
event_id	VARCHAR(190)	Sí	PK 1
created_at	VARCHAR(30)	Sí	-

product_specs

Tabla 35. Campos de product_specs

Campo	Tipo	Requerido	Clave / valor inicial
product_id	INTEGER	Sí	PK 1 FK products.id
spec_key	VARCHAR(80)	Sí	PK 2
spec_value	VARCHAR(500)	Sí	-

products

Tabla 36. Campos de products

Campo	Tipo	Requerido	Clave / valor inicial
id	INTEGER	Sí	PK 1
sku	VARCHAR(80)	Sí	-
category	VARCHAR(40)	Sí	FK categories.slug
name	VARCHAR(220)	Sí	-
brand	VARCHAR(90)	Sí	-
description	TEXT	Sí	-
image	VARCHAR(1500)	Sí	; inicial ''
source_url	VARCHAR(1500)	Sí	; inicial ''
source_checked	VARCHAR(20)	Sí	; inicial ''
price	INTEGER	Sí	; inicial 0
compare_price	INTEGER	Sí	; inicial 0
stock	INTEGER	Sí	; inicial 0
active	INTEGER	Sí	; inicial 1
featured	INTEGER	Sí	; inicial 0
verified	INTEGER	Sí	; inicial 0
supplier_id	INTEGER	No	FK suppliers.id
updated_at	VARCHAR(30)	Sí	-

rate_limits

Tabla 37. Campos de rate_limits

Campo	Tipo	Requerido	Clave / valor inicial
bucket	VARCHAR(64)	Sí	PK 1
attempts	INTEGER	Sí	-
expires_at	INTEGER	Sí	-

reset_tokens

Tabla 38. Campos de reset_tokens

Campo	Tipo	Requerido	Clave / valor inicial
token_hash	VARCHAR(64)	Sí	PK 1
user_id	INTEGER	Sí	FK users.id
expires_at	INTEGER	Sí	-

roles

Tabla 39. Campos de roles

Campo	Tipo	Requerido	Clave / valor inicial
code	VARCHAR(20)	Sí	PK 1
name	VARCHAR(60)	Sí	-

settings

Tabla 40. Campos de settings

Campo	Tipo	Requerido	Clave / valor inicial
setting_key	VARCHAR(80)	Sí	PK 1
setting_value	TEXT	Sí	-

suppliers

Tabla 41. Campos de suppliers

Campo	Tipo	Requerido	Clave / valor inicial
id	INTEGER	Sí	PK 1
name	VARCHAR(120)	Sí	-
email	VARCHAR(190)	Sí	-
phone	VARCHAR(30)	Sí	; inicial ''
active	INTEGER	Sí	; inicial 1
created_at	VARCHAR(30)	Sí	-

users

Tabla 42. Campos de users

Campo	Tipo	Requerido	Clave / valor inicial
id	INTEGER	Sí	PK 1
name	VARCHAR(120)	Sí	-
email	VARCHAR(190)	Sí	-
password	VARCHAR(255)	Sí	-
role	VARCHAR(20)	Sí	FK roles.code; inicial 'customer'
phone	VARCHAR(30)	Sí	; inicial ''
auth_version	INTEGER	Sí	; inicial 1
active	INTEGER	Sí	; inicial 1
created_at	VARCHAR(30)	Sí	-

16.5 Evidencia Git y contribuciones

Se adjunta el historial de cambios funcionales disponible; los cierres de documentación adicionales pueden consultarse en el respaldo Git incluido en el paquete. La primera entrada registra la versión existente al iniciar esta revisión. No es evidencia de desarrollo progresivo anterior del equipo. El equipo proporcionó el repositorio https://github.com/EESH02011/NEXTECH-. Este enlace no acredita por sí solo los aportes históricos. La actualización documental no configura el remoto local ni publica cambios.

8e93d4a | 2026-10-06T23:46:26-06:00 | NexTech automation | fix: normaliza especificaciones y verifica formularios y datos demo

c3a02c0 | 2026-10-06T23:31:46-06:00 | NexTech automation | feat: incorpora REST, CRUD administrativos, reportes e integridad de inventario

efa5eaa | 2026-10-06T23:20:45-06:00 | NexTech automation | chore: registra la version existente antes de la revision academica

Tabla 43. Aportaciones del equipo por completar

Integrante	Aporte real / evidencia	Commit o entregable
Soto Saavedra Erwin Emmanuel	Por documentar por el integrante	Pendiente de evidencia

Tabla 43. Aportaciones del equipo por completar

Integrante	Aporte real / evidencia	Commit o entregable
Aguirre Sierra Juan Damián	Por documentar por el integrante	Pendiente de evidencia
Crisanto Vázquez Sofía	Por documentar por el integrante	Pendiente de evidencia
Rosales Alarcón Amairani	Por documentar por el integrante	Pendiente de evidencia
Gutiérrez Rocha Kevin Yahir	Por documentar por el integrante	Pendiente de evidencia
Rivera Alvarado Itzel Daniela	Por documentar por el integrante	Pendiente de evidencia

16.6 Manual breve y fuentes de evidencia

Cliente: abrir catálogo, filtrar, agregar al carrito, iniciar sesión, verificar cantidades, confirmar compra y consultar Mis pedidos. Administrador: entrar a Administración, revisar Productos/Categorías/Proveedores/Usuarios, gestionar estados en Ventas/pedidos, comprobar Inventario y consultar Reportes. Para una prueba sin cobro mantener demostración activa. Las cuentas demo y sus contraseñas se generan con scripts/demo.php en una base nueva.

Tabla 44. Archivos y fuentes

Archivo	Contenido
Requerimientos_Proyectos_Finales_Software.md	Documento proporcionado por el usuario: 35 requisitos y alcance POS.
Lineamientos_Documentacion_Proyecto_Final_Software.pdf	Documento proporcionado por el usuario: estructura obligatoria y evidencia.
docs/api.md / api-routes.json	Contratos de las 56 rutas.
database/schema.sql / seed.sql	Creación relacional y catálogo reproducible sin clientes reales.
docs/database.md / database-model.json	Diccionario extraído de la base creada.
docs/diagrams/*.mmd	Modelo ER completo y arquitectura editables.
test/integration.mjs / docs/evidencias/pruebas.json	Casos HTTP ejecutables y resultado observado.
docs/evidencias/*.png / git-historial.txt	Capturas reales locales y log de esta revisión.
CATALOGO-FUENTES.csv / private/catalog.json	Referencias oficiales por modelo y fecha de consulta ya registrada en el catálogo.
README.md / VALIDACION.md	Instalación, configuración, alcance y límites de validación.

Las ilustraciones y esquemas de este PDF fueron preparados para explicar la implementación; los wireframes reconstruidos se identifican como tales. Las capturas corresponden a instalaciones locales con datos de demostración. Las cifras de pruebas provienen de la ejecución registrada, no de objetivos esperados.